2-1

LangGraph 정통 이론 & 고급 기능 실습

학습 목차

- **LangGraph의 필요성**
  - Cyclic Graph와 반복 추론
  - LangChain과 LangGraph 비교
- **그래프 구성 요소**
  - Node, Edge, State
  - State Schema - TypedDict, Pydantic
  - Reducer
- **안정적인 Agent 실행**
  - Checkpointer와 상태 영속화
  - Human-in-the-loop

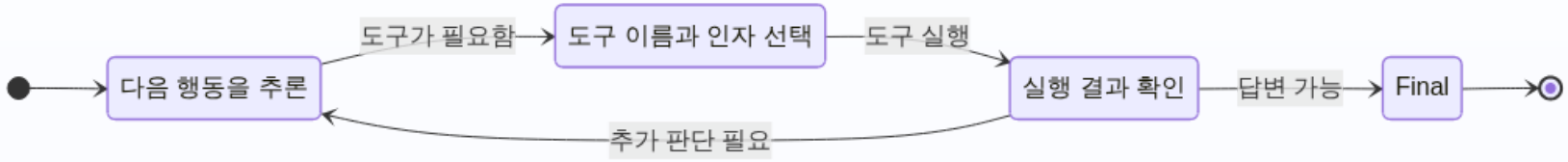
© Scopelabs. All Rights Reserved

LangGraph의 필요성

Cyclic Graph와 반복 추론

Agent는 한 번의 모델 호출로 끝나지 않고 판단, 도구 사용, 관찰을 반복해야 하는 경우가 많다.

- 검색이 부족하면 다시 검색
- 도구 실행이 실패하면 다른 경로를 선택
- 답변 품질이 낮으면 스스로 검토 및 보안



이런 반복 흐름은 Cyclic Graph의 형태로 표현 가능하다.

LangChain과 LangGraph 비교

LangChain은 LLM, 프롬프트, 도구, 파서를 연결하는 체인 구성에 강하다.

- 입력에서 출력까지 비교적 선형적인 흐름에 적합
- LCEL을 통해 실행 단위를 간결하게 조합
- 명확한 파이프라인에 유리

LangGraph는 상태를 가진 반복 워크플로우와 분기 흐름을 설계하는 데 강하다.

- 조건부 라우팅으로 다음 실행 노드를 선택
- 중단, 재개, 승인, 재시도 같은 서비스 요구사항을 구조화한다.
- 복잡한 Agent를 추적 가능한 실행 그래프로 표현
- 작업 상태를 그래프 전체에서 공유

그래프 구성 요소

Node, Edge, State (1)

LangGraph의 기본 요소는 Node, Edge, State

- Node: 실제 작업을 수행하는 실행 단위
- Edge: 한 노드가 끝난 뒤 이동할 노드를 정의
- State: 그래프 실행 중 유지되는 공유 데이터

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class AgentState(TypedDict):
    question: str
    answer: str

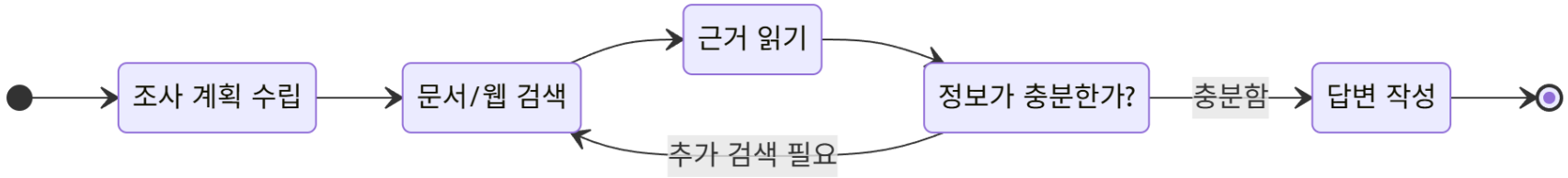
def call_llm(state: AgentState) -> AgentState:
    return {"answer": f"질문에 답변: {state['question']}"}

graph = StateGraph(AgentState)
graph.add_node("llm", call_llm)
```

Node, Edge, State (2)

Agent 그래프는 상태를 읽고 갱신하는 노드들의 연결로 이해할 수 있다.

- LLM 호출 노드는 다음 행동을 판단한다.
- Tool Node는 선택된 도구를 실행하고 결과를 상태에 기록한다.
- 조건부 Edge는 도구를 더 사용할지, 최종 답변으로 갈지 결정한다.



State Schema - TypedDict, Pydantic

State Schema는 그래프가 어떤 데이터를 들고 실행되는지 정의한다.

- TypedDict는 가볍고 직관적인 상태 타입 정의에 적합하다.
- Pydantic은 검증과 기본값, 설명이 필요한 상태 정의에 유리하다.
- 명확한 상태 스키마는 노드 간 데이터 계약을 안정적으로 만든다.

```
from typing import TypedDict
from pydantic import BaseModel, Field

class AgentState(TypedDict):
    question: str
    documents: list[str]
    answer: str

class AgentStateModel(BaseModel):
    question: str = Field(description="사용자 질문")
    documents: list[str] = Field(default_factory=list)
    answer: str ¦ None = None
```

Reducer

Reducer는 여러 노드가 같은 상태 필드를 갱신할 때 값을 합치는 규칙이다.

- Reducer가 명확해야 그래프 실행 결과를 예측할 수 있다.
  - 메시지 목록은 새 메시지를 기존 목록에 누적해야 함
  - 점수나 카운터는 더하거나 최신 값으로 교체할 수 있음
- 병렬 실행에서는 여러 결과를 하나의 상태로 합치는 규칙이 특히 중요하다.

```
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]

# 여러 노드가 steps를 반환하면 기존 목록 뒤에 누적된다.
```

안정적인 Agent 실행

Checkpointer와 상태 영속화

Checkpointer는 그래프 실행 상태를 저장해 중단과 재개를 가능하게 하는 기능이다.

- Agent 작업이 길어지면 중간에 실패하거나, 추가 입력이 필요한 경우가 존재.
- 이를 위해 상태를 저장하기 위한 기능
  - 대화형 서비스에서는 사용자별 세션 상태를 유지

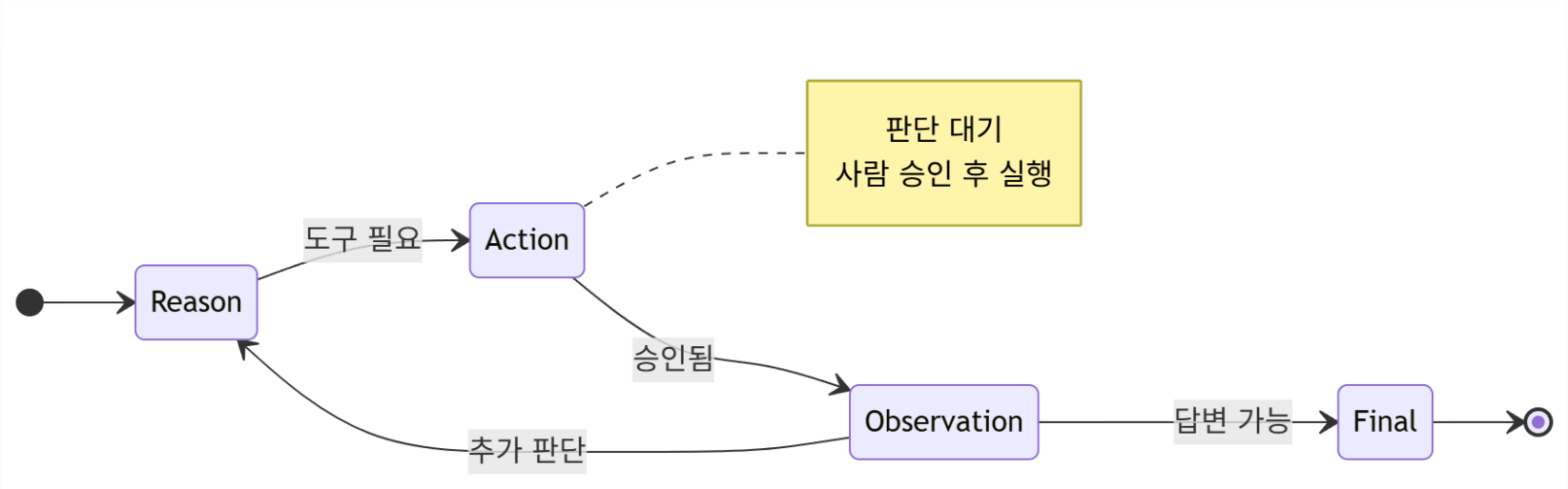
안정적인 서비스를 위해 필수

- 장애가 발생 시 이전 단계까지의 실행 기록을 복원
- 승인 대기, 외부 시스템 응답 대기 같은 비동기 흐름 제어
- 실행 이력을 추적해 디버깅과 평가에 활용

Human-in-the-loop (1)

Human-in-the-loop은 Agent 실행 중 특정 지점에서 사람의 판단을 개입시키는 설계이다.

- 민감한 도구 실행 전에 승인을 받을 수 있다.
- 모델의 판단이 불확실할 때 사용자에게 선택지를 요청할 수 있다.
- 자동화의 효율성과 사람의 통제권을 함께 확보한다.



Human-in-the-loop (2)

사람의 개입 지점은 위험도와 비용을 기준으로 설계한다.

- 외부 발송, 결제, 삭제처럼 되돌리기 어려운 행동
- 정보가 부족한 경우에는 추측하지 않고 질문
- 반복 승인으로 흐름이 느려지지 않도록 승인 기준을 명확히
- 승인 결과도 상태에 기록
  - 이후 실행이 같은 결정을 참조